

React Basics

A quick-reference cheat sheet for getting started with React

1. What is React?

React is a JavaScript library for building user interfaces, created and maintained by Meta. It lets you build encapsulated, reusable pieces of UI called **components**, and efficiently updates the DOM when your data changes using a **virtual DOM** diffing process.

- Declarative: you describe *what* the UI should look like, React handles the *how*.
- Component-based: build encapsulated components that manage their own state.
- Learn once, write anywhere: works for web (React DOM) and mobile (React Native).

2. Setting Up a Project

The fastest way to start a new React app today is with **Vite**:

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

This spins up a dev server with hot module reloading. Other common tools include Next.js (full framework with routing/SSR) and Create React App (legacy, no longer recommended).

3. JSX

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. It compiles down to `React.createElement()` calls.

```
function Greeting() {
  const name = "Sam";
  return <h1>Hello, {name}!</h1>;
}
```

- Curly braces `{ }` embed JavaScript expressions.
- JSX elements must return a single parent element (or use a Fragment `<> . . . </>`).
- Use `className` instead of `class`.

4. Components

Components are reusable, self-contained pieces of UI. Modern React almost always uses **function components**.

```
function Welcome({ name }) {
```

```

    return <p>Welcome back, {name}!</p>;
  }

  // Usage
  <Welcome name="Asha" />

```

Component names must start with a capital letter so React can distinguish them from HTML tags.

5. Props

Props ("properties") are how data flows from a parent component into a child. They are read-only from the child's perspective.

```

function Avatar({ src, alt }) {
  return <img src={src} alt={alt} className="avatar" />;
}

<Avatar src="/me.png" alt="My profile photo" />

```

6. State with useState

State lets a component hold and update its own data over time. Updating state triggers a re-render.

```

import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}

```

Note: Never mutate state directly (e.g. `count++`). Always use the setter function returned by `useState`.

7. Handling Events

React events are named using camelCase and you pass a function reference, not a string.

```

function Form() {
  function handleSubmit(e) {
    e.preventDefault();
    console.log("Submitted!");
  }

  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">Send</button>
    </form>
  );
}

```

8. Rendering Lists

Use `.map()` to turn an array of data into an array of elements. Each item needs a unique key prop.

```
const fruits = ["Apple", "Banana", "Cherry"];

function FruitList() {
  return (
    <ul>
      {fruits.map((fruit) => (
        <li key={fruit}>{fruit}</li>
      ))}
    </ul>
  );
}
```

9. Conditional Rendering

```
function Status({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? <p>Welcome back!</p> : <p>Please log in.</p>}
    </div>
  );
}
```

- Ternary condition `? a : b` — for either/or rendering.
- Logical AND condition `&&` `<Component />` — render only if true.
- Early return `null`; — render nothing at all.

10. Side Effects with `useEffect`

`useEffect` lets you run code after render — for data fetching, subscriptions, timers, or manually changing the DOM.

```
import { useState, useEffect } from "react";

function Clock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setTime(new Date()), 1000);
    return () => clearInterval(id); // cleanup
  }, []); // empty array = run once on mount

  return <p>{time.toLocaleTimeString()}</p>;
}
```

Note: The dependency array controls when the effect re-runs. Omitting it runs the effect after every render; an empty array runs it only once.

11. Common Built-in Hooks

Hook	Purpose
useState	Add local state to a component
useEffect	Run side effects after render
useContext	Read a value from React Context
useRef	Persist a mutable value / reference a DOM node
useMemo	Memoize an expensive computed value
useCallback	Memoize a function reference
useReducer	Manage complex state via a reducer function

12. Composing Components

Components can be nested and combined using `children` to build complex UIs from simple pieces.

```
function Card({ children }) {  
  return <div className="card">{children}</div>;  
}  
  
function App() {  
  return (  
    <Card>  
      <h2>Title</h2>  
      <p>Some card content goes here.</p>  
    </Card>  
  );  
}
```

13. Quick Tips & Best Practices

- Keep components small and focused on one responsibility.
- Lift state up to the closest common ancestor when multiple components need it.
- Never call hooks conditionally or inside loops — call them at the top level only.
- Use the React DevTools browser extension to inspect component trees and state.
- Prefer derived state (computed during render) over duplicating data in multiple `useState` calls.